

Web Scraping (rvest & Beautiful Soup)

Lecture 13

Dr. Colin Rundel



Hypertext Markup Language

HTML is the language web pages are written in. It describes a page as nested elements marked by tags, which the browser renders into what we see. Web scraping is the reverse process, recovering the underlying data from that markup.

```
1 <html>
2   <head>
3     <title>This is a title</title>
4   </head>
5   <body>
6     <p align="center">Hello world!</p>
7     <br/>
8     <div class="name" id="first">John</div>
9     <div class="name" id="last">Doe</div>
10    <div class="contact">
11      <div class="home">555-555-1234</div>
12      <div class="home">555-555-2345</div>
13      <div class="work">555-555-9999</div>
14      <div class="fax">555-555-8888</div>
15    </div>
16  </body>
17 </html>
```

Anatomy of an element

An HTML document is built from elements. Most have an opening tag, optional attributes, content, and a closing tag. Void elements such as `br` and `img` have no content or closing tag.

```
1 <div class="name" id="first">John</div>
```

Part	Here	Notes
tag name	<code>div</code>	what kind of element this is (<code>p</code> , <code>a</code> , <code>table</code> , <code>div</code> , ...)
attributes	<code>class="name"</code> <code>id="first"</code>	name and value pairs, any number of them
content	<code>John</code>	text, other elements, or both

Two attributes matter more than the rest when scraping:

- `class` - used to label a group of similar elements. Multiple elements can share a class, and one element can have multiple classes (`class="name bold"`).
- `id` - labels a single element and *should* be unique within a page.

The document tree

Elements nest inside one another, so a page is a tree. Selectors describe elements by both what they are and by where they sit in the tree.

```
html
├── head
│   └── title
└── body
    ├── p
    ├── br
    ├── div.name#first
    ├── div.name#last
    └── div.contact
        ├── div.home
        ├── div.home
        ├── div.work
        └── div.fax
```

- `html` is the root, an *ancestor* of everything else
- `body` is the *parent* of `p`, and `p` is a *child* of `body`
- `p`, `br` and the first three `div`s are *siblings*, since they share a *parent*
- `div.home` is a *descendant* of `body` but not a *child* of it

rvest

`rvest` is a package in the tidyverse that makes processing and manipulation of HTML data straightforward. It provides high-level functions for interacting with HTML via the `xml2` package.

- `read_html()` - read HTML data from a URL or character string.
- `html_elements()` / ~~`html_nodes()`~~ - select specified elements from the HTML document using CSS selectors (or XPath).
- `html_element()` / ~~`html_node()`~~ - select the first match within each input element using CSS selectors (or XPath).
- `html_table()` - parse an HTML table into a data frame.
- `html_text()` / `html_text2()` - extract a tag's text content.
- `html_name()` - extract a tag/element's name(s).
- `html_attrs()` - extract all attributes.
- `html_attr()` - extract attribute value(s) by name.

HTML, rvest, & xml2

```
1 html =
2 '<html>
3   <head>
4     <title>This is a title</title>
5   </head>
6   <body>
7     <p align="center">Hello world!</p>
8     <br/>
9     <div class="name" id="first">John</div>
10    <div class="name" id="last">Doe</div>
11    <div class="contact">
12      <div class="home">555-555-1234</div>
13      <div class="home">555-555-2345</div>
14      <div class="work">555-555-9999</div>
15      <div class="fax">555-555-8888</div>
16    </div>
17  </body>
18 </html>'
```

```
1 read_html(html)
```

```
{html_document}
<html>
[1] <head>\n<meta http-equiv="Content-Type" cont
[2] <body>\n    <p align="center">Hello world!</
```

```
1 read_html(html)[1]
```

```
$node
<pointer: 0x7bfd8ec780>
```

Selecting elements

`html_element()` returns the first element matching a selector.

```
1 read_html(html) |> html_element("p")
```

```
{html_node}  
<p align="center">
```

```
1 read_html(html) |> html_element("p") |> html_text()
```

```
[1] "Hello world!"
```

```
1 read_html(html) |> html_element("p") |> html_name()
```

```
[1] "p"
```

```
1 read_html(html) |> html_element("p") |> html_attrs()
```

```
align  
"center"
```

```
1 read_html(html) |> html_element("p") |> html_attr("align")
```

```
[1] "center"
```

Selecting multiple tags

`html_elements()` returns every match as a node set, extraction functions are vectorized over the node set.

```
1 read_html(html) |> html_elements("div")
```

R

```
{xml_nodeset (7)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...  
[4] <div class="home">555-555-1234</div>  
[5] <div class="home">555-555-2345</div>  
[6] <div class="work">555-555-9999</div>  
[7] <div class="fax">555-555-8888</div>
```

```
1 read_html(html) |> html_elements("div") |> html_text()
```

R

```
[1] "John"  
[2] "Doe"  
[3] "\n      555-555-1234\n      555-555-2345\n      555-555-9999\n      555-555-8888\n      "  
[4] "555-555-1234"  
[5] "555-555-2345"  
[6] "555-555-9999"  
[7] "555-555-8888"
```

CSS selectors

CSS (Cascading Style Sheets) is the language used to style web pages. A stylesheet is a list of rules, and each rule starts with a selector that says which elements the styling applies to.

```
1 div.name {  
2   color: blue;  
3   font-weight: bold;  
4 }  
5  
6 #first {  
7   font-size: 120%;  
8 }
```

Scraping tools borrow just the selector half of the language. It is a compact, standardized way of describing a set of elements, and it is already understood by a wide variety of tools.

Basic selectors

The simplest selectors match on an element's tag name, its class, or its id. These can be written together with no spaces to require that all of them match, or separated by commas to match any of them.

Selector	Example	Description
element	<code>p</code>	Select all <code><p></code> elements
.class	<code>.name</code>	Select all elements with <code>class="name"</code>
#id	<code>#first</code>	Select all elements with <code>id="first"</code>
element.class	<code>div.name</code>	Select all <code><div></code> elements with <code>class="name"</code>
A, B	<code>p, #last</code>	Select everything matched by either selector

```
1 read_html(html) |> html_elements(".name")
```

```
{xml_nodeset (2)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>
```

```
1 read_html(html) |> html_elements("p, #last")
```

```
{xml_nodeset (2)}  
[1] <p align="center">Hello world!</p>  
[2] <div class="name" id="last">Doe</div>
```

```
1 read_html(html) |> html_elements("#first")
```

```
{xml_nodeset (1)}  
[1] <div class="name" id="first">John</div>
```

Combinators

Combinators join two selectors and match on the relationship between elements in the tree. The element that is returned is always the last one in the selector.

Selector	Example	Description
A B	<code>body div</code>	Select all <code><div></code> elements anywhere inside <code><body></code> (descendant)
A > B	<code>body > div</code>	Select all <code><div></code> elements directly inside <code><body></code> (child)
A + B	<code>.work + div</code>	Select the <code><div></code> immediately after a <code>.work</code> element (adjacent sibling)
A ~ B	<code>.home ~ div</code>	Select all <code><div></code> siblings that come after a <code>.home</code> element

```
1 read_html(html) |> html_elements("body div")  
  
{xml_nodeset (7)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-  
[4] <div class="home">555-555-1234</div>  
[5] <div class="home">555-555-2345</div>  
[6] <div class="work">555-555-9999</div>  
[7] <div class="fax">555-555-8888</div>
```

```
1 read_html(html) |> html_elements("body > div")  
  
{xml_nodeset (3)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>  
[3] <div class="contact">\n      <div class="home">555-  
  
1 read_html(html) |> html_elements(".work + div")  
  
{xml_nodeset (1)}  
[1] <div class="fax">555-555-8888</div>
```

Attribute selectors

Any attribute can be used in a selector. This is most useful for links and images, e.g. `a[href^="https"]` or `img[src$=".png"]`.

Selector	Example	Description
<code>[attr]</code>	<code>[align]</code>	Select all elements with an align attribute
<code>[attr=value]</code>	<code>[class=fax]</code>	Select all elements where class is exactly “fax”
<code>[attr^=value]</code>	<code>[class^=ho]</code>	Select all elements where class starts with “ho”
<code>[attr\$=value]</code>	<code>[id\$=st]</code>	Select all elements where id ends with “st”
<code>[attr*=value]</code>	<code>[class*=or]</code>	Select all elements where class contains “or”

```
1 read_html(html) |> html_elements("[align]")
```

```
{xml_nodeset (1)}  
[1] <p align="center">Hello world!</p>
```

```
1 read_html(html) |> html_elements("[class^=ho]")
```

```
{xml_nodeset (2)}  
[1] <div class="home">555-555-1234</div>  
[2] <div class="home">555-555-2345</div>
```

```
1 read_html(html) |> html_elements("[id$=st]")
```

```
{xml_nodeset (2)}  
[1] <div class="name" id="first">John</div>  
[2] <div class="name" id="last">Doe</div>
```

Pseudo-classes

Pseudo-classes start with a `:` and select on an element's position among its siblings or on a condition that the other selectors cannot express.

Selector	Example	Description
<code>:first-child</code>	<code>li:first-child</code>	Select <code></code> elements that are the first child of their parent
<code>:last-child</code>	<code>li:last-child</code>	Select <code></code> elements that are the last child of their parent
<code>:nth-child(n)</code>	<code>tr:nth-child(2)</code>	Select <code><tr></code> elements that are the second child of their parent
<code>:nth-child(odd)</code>	<code>tr:nth-child(odd)</code>	Select every other <code><tr></code> , also <code>even</code> and formulas like <code>3n+1</code>
<code>:not(A)</code>	<code>div:not(.ad)</code>	Select <code><div></code> elements that do not match <code>.ad</code>
<code>:has(A)</code>	<code>div:has(img)</code>	Select <code><div></code> elements that contain an <code></code>

Not every tool supports every pseudo-class, so a selector that works in the browser may fail in a scraping library. For example, the newer `details:open` is an error in `rvest` but works in `Beautiful Soup`.

Pseudo-classes

```
1 read_html(html) |> html_elements(".contact div:first-child")
```

R

```
{xml_nodeset (1)}  
[1] <div class="home">555-555-1234</div>
```

```
1 read_html(html) |> html_elements(".contact div:nth-child(2)")
```

R

```
{xml_nodeset (1)}  
[1] <div class="home">555-555-2345</div>
```

```
1 read_html(html) |> html_elements(".contact div:not(.home)")
```

R

```
{xml_nodeset (2)}  
[1] <div class="work">555-555-9999</div>  
[2] <div class="fax">555-555-8888</div>
```

```
1 read_html(html) |> html_elements("div:has(.work)")
```

R

```
{xml_nodeset (1)}  
[1] <div class="contact">\n      <div class="home">555-555-1234</div>\n      ...
```

Choosing a selector

There are usually many selectors that return the same elements. The goal is one that keeps working even if the page changes.

- Prefer classes and IDs that describe the content (`.price`, `#reviews`) over ones that describe position (`div > div:nth-child(3)`).
- Be as specific as needed and no more. A long chain of tags breaks the first time the site adds a `div`.
- Be wary of machine-generated class names like `.css-1q2w3e`, which tend to change with every update of the site.
- Anchor on a stable container, then select within it (`#results .title`).
- Always check how many elements came back. Too many or too few is the first sign that the selector is wrong.
- Not everything needs to be purely selector based - you can also process via R or Python after extraction.

html_text() vs html_text2()

```
1 par = read_html(  
2   "<p>  
3     First sentence.  
4     Same line.<br>New line.  
5   </p>"  
6 )
```

```
1 par |> html_text()
```

```
[1] "\n    First sentence.\n    Same line.New line.\n "
```

```
1 par |> html_text() |> cat(sep="\n")
```

```
First sentence.  
Same line.New line.
```

```
1 par |> html_text2()
```

```
[1] "First sentence. Same line.\nNew line."
```

```
1 par |> html_text2() |> cat(sep="\n")
```

```
First sentence. Same line.  
New line.
```

HTML tables

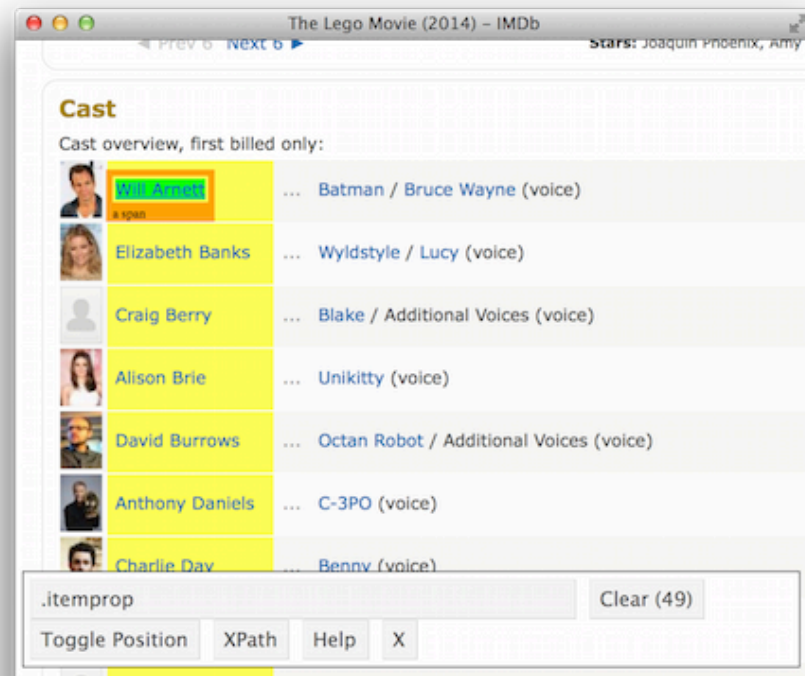
```
1 html_table =
2 '<html>
3   <head>
4     <title>This is a title</title>
5   </head>
6   <body>
7     <table>
8       <tr> <th>a</th> <th>b</th> <th>c</th> </tr>
9       <tr> <td>1</td> <td>2</td> <td>3</td> </tr>
10      <tr> <td>2</td> <td>3</td> <td>4</td> </tr>
11      <tr> <td>3</td> <td>4</td> <td>5</td> </tr>
12    </table>
13  </body>
14 </html>'
```

```
1 read_html(html_table) |>
2   html_elements("table") |>
3   html_table()
```

```
[[1]]
# A tibble: 3 × 3
      a     b     c
<int> <int> <int>
1     1     2     3
2     2     3     4
3     3     4     5
```

SelectorGadget

This is a JavaScript-based tool that helps you interactively build an appropriate CSS selector for the content you are interested in.



selectorgadget.com

Web scraping considerations

“Can you?” vs “Should you?”

Researchers just released profile data on 70,000 OkCupid users without permission

By Brian Resnick | @B_resnick | brian@vox.com | May 12, 2016, 6:00pm EDT

A group of researchers has released a data set on nearly 70,000 users of the online dating site OkCupid. The data dump breaks the cardinal rule of social science research ethics: **It took identifiable personal data without permission.**

The information — while publicly available to OkCupid users — was collected by Danish researchers who never contacted OkCupid or its clientele about using it.

The data, collected from November 2014 to March 2015, includes user names, ages, gender, religion, and personality traits, as well as answers to the personal questions the site asks to help match potential mates. The users hail from a few dozen countries around the world.

The data dump did not reveal anyone's real name. But it's entirely possible to use clues from a user's location, demographics, and OkCupid user name to determine their identity.

If your OkC username is one you've used anywhere else, I now know your sexual preferences & kinks, your answers to thousands of questions.

— Scott B. Weingart (@scott_bot) May 11, 2016

“Can you?” vs “Should you?”



Emil OW Kirkegaard @KirkegaardEmil · May 8

The OKCupid paper has now been submitted. This means that the dataset is now public! Enjoy! :) [openpsych.net/forum/showthre...](https://openpsych.net/forum/showthread.php?p=10000)



26



38



Ethan Jewett @esjewett · May 11

@KirkegaardEmil This data set is highly re-identifiable. Even includes usernames? Was any work at all done to anonymize it?



3



9



Emil OW Kirkegaard

@KirkegaardEmil



Follow

@esjewett No. Data is already public.

Crawl rules & robots.txt

Sites use the [robots exclusion standard](#) to publish crawler rules in [robots.txt](#). Following these rules does not mean you are authorized to use the data. Also consider the site's terms and the privacy of the people represented in the data.

You can find examples at all of your favorite websites: [Google](#), [Facebook](#), etc.

These files are meant to be machine readable, so the [polite](#) package can handle this for us (and much more).

```
1 polite::bow("https://www.google.com")  
2 polite::bow("https://www.facebook.com")
```

R

Scraping with polite

Beyond the `bow()` function, `polite` also has a `scrape()` function that helps you scrape a website while maintaining the three pillars of a polite session:

- seek permission
- take it slowly
- never ask twice

This is achieved by using the session object from `bow()` within the `scrape()` function to make the request (this is equivalent to `rvest`'s `read_html()` and returns a parsed HTML object).

New paths within the website can be accessed by using the `nod()` function before using `scrape()`.

Rate limiting

Making requests too quickly can overload a server, get your IP blocked, or violate a site's terms of service. Adding delays between requests is essential for responsible scraping.

The simplest approach is `Sys.sleep()` between requests.

```
1 for (url in urls) {  
2   read_html(url) |> html_elements(".title")  
3   Sys.sleep(1) # wait 1 second between requests  
4 }
```

`polite` uses the larger of the requested delay (5 seconds by default) and the crawl delay in `robots.txt`.

```
1 session = polite::bow("https://example.com")  
2  
3 polite::nod(session, "page/1") |> polite::scrape()  
4 polite::nod(session, "page/2") |> polite::scrape()
```

JavaScript rendered pages

`read_html()` only sees the static HTML delivered by the server. Pages that use JavaScript to render content will appear empty or incomplete.

`rvest::read_html_live()` runs Chrome via `chromote` and returns a `LiveHTML` object. It lets us select dynamically generated elements, though some content may require waiting, scrolling, or clicking first.

```
1 page = read_html_live("https://example.com")
2 page |> html_elements(".some-js-rendered-class")
```

The live object supports browser interactions (these selectors are placeholders).

```
1 page$click("#load-more-button")
2 page$scroll_to(top = 1000)
3 page$type("#search-box", "query text")
4 page$view()
5 page$session$screenshot("screenshot.png")
```

Graceful error handling

When scraping many pages, retrieval or parsing can fail. Wrapping `read_html()` lets us capture those errors and continue.

`purrr::possibly()` wraps a function so that errors return a default value instead of stopping.

```
1 safe_read = purrr::possibly(read_html, otherwise = NULL)
2 urls |>
3   purrr::map(safe_read)
```

R

`purrr::safely()` is similar but returns a list with `result` and `error` components, letting you inspect what went wrong.

```
1 safe_read = purrr::safely(read_html)
2 results = urls |> purrr::map(safe_read)
```

R

```
1 results |> purrr::map("result") # successful results (NULL on failure)
2 results |> purrr::map("error")  # error objects (NULL on success)
```

R

Missing selectors often return empty results instead of errors. Check counts and required fields explicitly. These

Live exercise - Rotten Tomatoes

Open [Movies at Home](#), sorted by popularity. Together, inspect the page and choose selectors for the first 20 movies displayed.

Build a data frame with one row per movie:

- title, Tomatometer score, displayed critic-status label, and absolute movie URL
- keep movies with missing scores or labels; use [NA](#) for missing fields

Visit each movie's URL to add its content rating, runtime, and audience-rating count. Preserve counts reported as ranges or thresholds (such as [1,000+](#)) as text.

Record the retrieval date and listing order. Check crawl rules and access before making requests, and use a delay

Beautiful Soup

Beautiful Soup

Beautiful Soup is the most widely used Python package for parsing and navigating HTML. It is installed as `beautifulsoup4` and imported as `bs4`.

- `BeautifulSoup()` - parse HTML from a string, returning a `BeautifulSoup` object.
- `select()` - select all matching elements using a CSS selector, returned as a list of `Tag` objects.
- `select_one()` - select the first matching element using a CSS selector.
- `find_all()` / `find()` - select elements by tag name and attribute values rather than a CSS selector.
- `get_text()` - extract a tag's text content.
- `.name` - a tag's name.
- `.attrs` - all of a tag's attributes as a dictionary.
- `tag["attr"]` / `tag.get("attr")` - extract an attribute value by name.

Beautiful Soup only parses HTML. It does not download pages or read tables. We will pair it with `requests` to download

HTML & bs4

```
1 html = '''<html>
2   <head>
3     <title>This is a title</title>
4   </head>
5   <body>
6     <p align="center">Hello world!</p>
7     <br/>
8     <div class="name" id="first">John</div>
9     <div class="name" id="last">Doe</div>
10    <div class="contact">
11      <div class="home">555-555-1234</div>
12      <div class="home">555-555-2345</div>
13      <div class="work">555-555-9999</div>
14      <div class="fax">555-555-8888</div>
15    </div>
16  </body>
17 </html>'''
```

py

```
1 from bs4 import BeautifulSoup
2
3 soup = BeautifulSoup(html, "html.parser")
4 type(soup)
```

py

```
<class 'bs4.BeautifulSoup'>
```

```
1 soup.title
```

py

```
<title>This is a title</title>
```

The second argument picks the parser. `"html.parser"` is built into Python. `"lxml"` is faster and `"html5lib"` is the most

Selecting elements

`select()` returns all matches as a list, and `select_one()` returns the first matching `Tag`, or `None`. Unlike `rvest`, these methods operate on one document or tag at a time.

```
1 soup.select("p")
```

py

```
[<p align="center">Hello world!</p>]
```

```
1 p = soup.select_one("p")
2 p
```

py

```
<p align="center">Hello world!</p>
```

```
1 p.get_text()
```

py

```
'Hello world!'
```

```
1 p.name
```

py

```
'p'
```

```
1 p.attrs
```

py

```
{'align': 'center'}
```

```
1 p["align"]
```

py

```
'center'
```

Many elements

The extraction methods belong to a single [Tag](#); they are not vectorized like rvest's `html_*()` functions. With more than one element we use a comprehension.

```
1 soup.select(".name").get_text()
```

py

AttributeError: ResultSet object has no attribute "get_text". You're probably treating a list of elements like a single element. Did you call `find_all()` when you meant to call `find()`?

```
1 [d.get_text() for d in soup.select(".name")]
```

py

```
['John', 'Doe']
```

```
1 [d["id"] for d in soup.select(".name")]
```

py

```
['first', 'last']
```

Nesting tags

```
1 soup.select("body div")
```

py

```
[<div class="name" id="first">John</div>, <div class="name" id="last">Doe</div>, <div class="contact" id="home">555-555-1234</div>, <div class="contact" id="home">555-555-2345</div>, <div class="contact" id="work">555-555-9999</div>, <div class="contact" id="fax">555-555-8888</div>]
```

```
1 soup.select("body > div")
```

py

```
[<div class="name" id="first">John</div>, <div class="name" id="last">Doe</div>, <div class="contact" id="home">555-555-1234</div>, <div class="contact" id="home">555-555-2345</div>, <div class="contact" id="work">555-555-9999</div>, <div class="contact" id="fax">555-555-8888</div>]
```

```
1 soup.select("body div div")
```

py

```
[<div class="contact" id="home">555-555-1234</div>, <div class="contact" id="home">555-555-2345</div>, <div class="contact" id="work">555-555-9999</div>]
```

Other selectors

The selector syntax is the same as before, since CSS selectors are a web standard and not part of either package.

```
1 soup.select("div.name")
```

py

```
[<div class="name" id="first">John</div>, <div class="name" id="last">Doe</div>]
```

```
1 soup.select("#first")
```

py

```
[<div class="name" id="first">John</div>]
```

```
1 soup.select("[align]")
```

py

```
[<p align="center">Hello world!</p>]
```

```
1 soup.select(".contact div")
```

py

```
[<div class="home">555-555-1234</div>, <div class="home">555-555-2345</div>, <div class="work">555-555-3456</div>]
```

Missing elements

When nothing matches, `select()` returns an empty list and `select_one()` returns `None`, so chaining a method onto a missing element is an error.

```
1 soup.select("span")
```

py

```
[]
```

```
1 print(soup.select_one("span"))
```

py

```
None
```

```
1 soup.select_one("span").get_text()
```

py

```
AttributeError: 'NoneType' object has no attribute 'get_text'
```

Attributes behave the same way: `tag["attr"]` raises a `KeyError` if the attribute is missing while `tag.get("attr")` returns `None`.

```
1 print(p.get("class"))
```

py

```
None
```

find() and find_all()

Beautiful Soup also has its own search methods that match on tag name and attribute values. `find_all()` returns a list and `find()` returns the first match (or `None`).

```
1 soup.find_all("div", class_="home")
```

py

```
[<div class="home">555-555-1234</div>, <div class="home">555-555-2345</div>]
```

```
1 soup.find(id="last")
```

py

```
<div class="name" id="last">Doe</div>
```

Searches can be chained, as each `Tag` can be searched in the same way as the full document.

```
1 soup.find("div", class_="contact").find_all("div", class_="work")
```

py

```
[<div class="work">555-555-9999</div>]
```

Extracting text

`get_text()` behaves like `html_text()` and returns the raw text, including the whitespace from the source. There is no direct equivalent of `html_text2()`, but the `separator` and `strip` arguments get us most of the way there.

```
1 contact = soup.select_one(".contact")
2 contact.get_text()
```

py

```
'\n555-555-1234\n555-555-2345\n555-555-9999\n555-555-8888\n'
```

```
1 contact.get_text(" ", strip=True)
```

py

```
'555-555-1234 555-555-2345 555-555-9999 555-555-8888'
```

```
1 list(contact.stripped_strings)
```

py

```
['555-555-1234', '555-555-2345', '555-555-9999', '555-555-8888']
```

Reading from the web

`BeautifulSoup()` does not accept a URL, so the page is downloaded first with `requests` and the text of the response is then parsed.

```
1 import requests
2
3 r = requests.get("https://example.com", timeout=10)
4 r.raise_for_status()
5
6 soup = BeautifulSoup(r.text, "html.parser")
7 soup.select(".title")
```

py

`raise_for_status()` turns a failed request (404, 500, ...) into an exception. Without it we would happily parse the error page.

Checking crawl rules

`urllib.robotparser` reads `robots.txt`. Check the rules for the same user agent that you will send with your requests.

```
1 from urllib.robotparser import RobotFileParser
2
3 user_agent = "sta523-example"
4 rp = RobotFileParser("https://www.google.com/robots.txt")
5 rp.read()
6
7 rp.can_fetch(user_agent, "https://www.google.com/search?q=duke")
8 rp.can_fetch(user_agent, "https://www.google.com/maps")
```

py

These rules apply to this site. Fetch a separate `robots.txt` when changing sites. An allowed path does not establish permission to reuse its data.

Rate limiting in Python

Use the site's crawl delay and request rate, when specified, with a conservative default. This loop uses only URLs on the site checked above.

```
1 import requests
2 import time
3
4 urls = ["https://www.google.com/search?q=duke",
5         "https://www.google.com/maps"]
6 delay = max(5, rp.crawl_delay(user_agent) or 0)
7 rate = rp.request_rate(user_agent)
8 if rate is not None:
9     delay = max(delay, rate.seconds / rate.requests)
10
11 for url in urls:
12     if rp.can_fetch(user_agent, url):
13         try:
14             r = requests.get(url, headers={"User-Agent": user_agent}, timeout=10)
15             r.raise_for_status()
16         finally:
17             time.sleep(delay)
```

py

Graceful error handling

For request failures, we can mimic `purrr::possibly()` with `try / except`, returning `None` on failure. Missing elements still need explicit checks.

```
1 def safe_read(url):
2     try:
3         r = requests.get(url, timeout=10)
4         r.raise_for_status()
5         return BeautifulSoup(r.text, "html.parser")
6     except requests.RequestException:
7         return None
8
9 pages = [safe_read(url) for url in urls]
```

py

Keeping the request exception gives us the same result/error structure as `purrr::safely()`.

```
1 def safe_read(url):
2     try:
3         r = requests.get(url, timeout=10)
4         r.raise_for_status()
5         return {"result": BeautifulSoup(r.text, "html.parser"), "error": None}
6     except requests.RequestException as e:
7         return {"result": None, "error": e}
```

py

Summary

rvest and BeautifulSoup

Task	rvest	Python
download a page	<code>read_html(url)</code>	<code>requests.get(url).text</code>
parse HTML	<code>read_html(html)</code>	<code>BeautifulSoup(html, "html.parser")</code>
select all matches	<code>html_elements(css)</code>	<code>select(css), find_all()</code>
first match per input	<code>html_element(css)</code>	<code>select_one(css), find()</code>
no match	empty node set / missing node	<code>[], None</code>
text	<code>html_text()</code> , <code>html_text2()</code>	<code>get_text()</code> , <code>get_text(sep, strip=True)</code>
tag name	<code>html_name()</code>	<code>.name</code>
all attributes	<code>html_attrs()</code>	<code>.attrs</code>
one attribute	<code>html_attr("href")</code>	<code>tag["href"], tag.get("href")</code>
tables	<code>html_table()</code>	<code>pandas.read_html()</code>
many elements	vectorized	comprehension
robots.txt	<code>polite::bow()</code>	<code>urllib.robotparser</code>
rate limiting	<code>polite::scrape()</code> , <code>Sys.sleep()</code>	<code>time.sleep()</code>
error handling	<code>possibly()</code> , <code>safely()</code>	<code>try / except</code>
JavaScript pages	<code>read_html_live()</code>	Playwright, Selenium

Takeaways

- HTML is hierarchical, and scraping is the work of picking the elements we want out of that hierarchy and flattening them into a data frame.
- CSS selectors do the picking in both languages. Tools like SelectorGadget and the browser's inspector help find a selector that matches what we want and nothing else.
- rvest's functions are vectorized over a set of elements, while BeautifulSoup's methods work on one [Tag](#) at a time and are combined with comprehensions.
- Just because a page can be scraped does not mean it should be. Check [robots.txt](#) and the terms of service, slow down your requests, and expect some of them to fail.